

CSE 3113 - Microprocessor and Assembly Lab

Lab Report

Md Sadek Hossain Asif

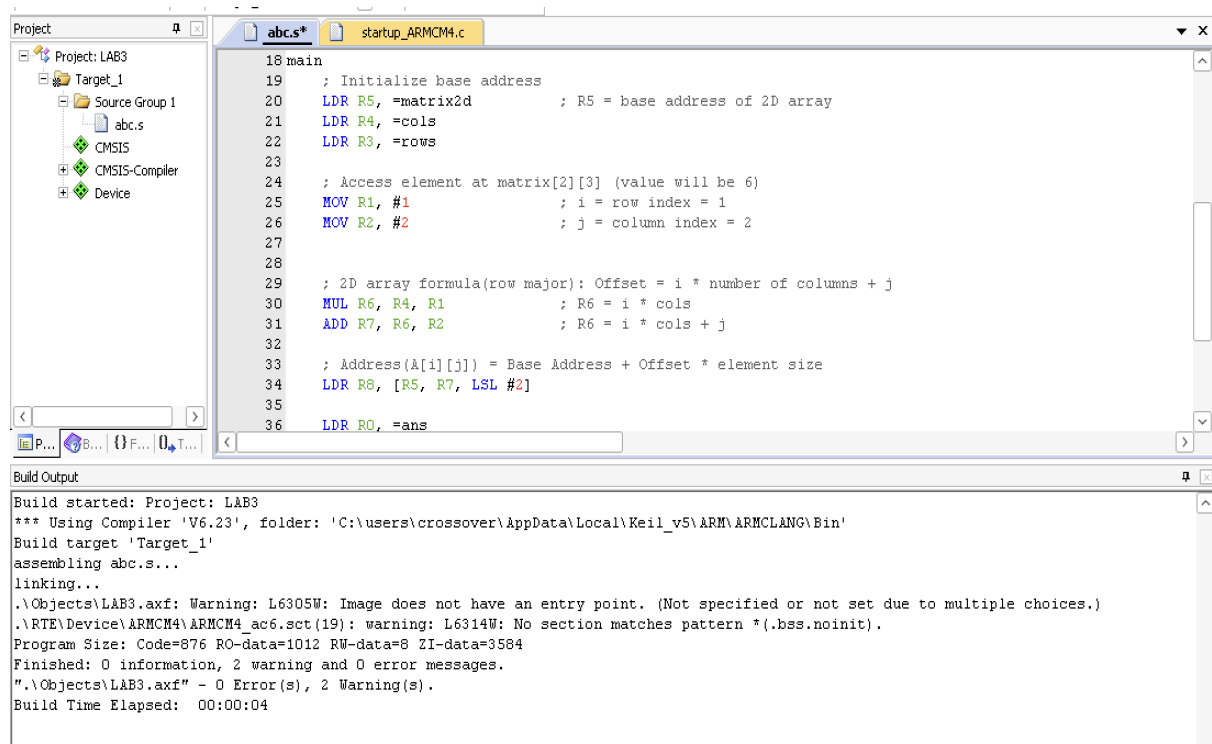
Class Roll: 15

June 26, 2025

1 Task 1

Create a 2D Array and use the 2D Array translation formula to access the Array elements

Build Status



The screenshot displays the Keil uVision IDE interface. The left pane shows the project structure for 'Project: LAB3', including 'Target_1', 'Source Group 1', and files 'abc.s' and 'startup_ARMCM4.c'. The main window shows the assembly code for 'abc.s' with the following content:

```
18 main
19 ; Initialize base address
20 LDR R5, =matrix2d ; R5 = base address of 2D array
21 LDR R4, =cols
22 LDR R3, =rows
23
24 ; Access element at matrix[2][3] (value will be 6)
25 MOV R1, #1 ; i = row index = 1
26 MOV R2, #2 ; j = column index = 2
27
28
29 ; 2D array formula(row major): Offset = i * number of columns + j
30 MUL R6, R4, R1 ; R6 = i * cols
31 ADD R7, R6, R2 ; R6 = i * cols + j
32
33 ; Address(A[i][j]) = Base Address + Offset * element size
34 LDR R8, [R5, R7, LSL #2]
35
36 LDR R0, =ans
```

The bottom pane shows the 'Build Output' window with the following text:

```
Build started: Project: LAB3
*** Using Compiler 'V6.23', folder: 'C:\users\crossover\AppData\Local\Keil_v5\ARM\ARMCLANG\Bin'
Build target 'Target_1'
assembling abc.s...
linking...
.\Objects\LAB3.axf: Warning: L6305W: Image does not have an entry point. (Not specified or not set due to multiple choices.)
.\RTE\Device\ARMCM4\ARMCM4.ac6.sct(19): warning: L6314W: No section matches pattern *(.bss.noinit).
Program Size: Code=876 RO-data=1012 RW-data=8 ZI-data=3584
Finished: 0 information, 2 warning and 0 error messages.
".\Objects\LAB3.axf" - 0 Error(s), 2 Warning(s).
Build Time Elapsed: 00:00:04
```

Figure 1: State of the system after the project is built.

Register Status After Execution

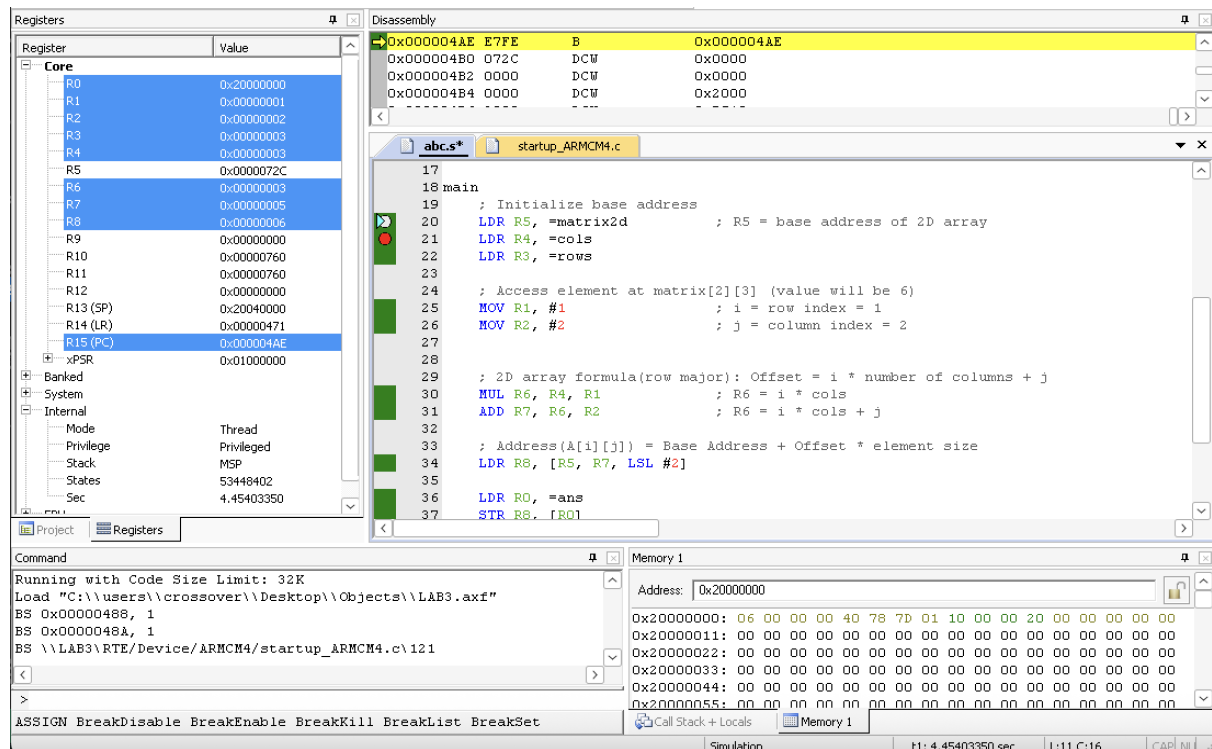


Figure 2: State of the system after the code has been executed.

2 Task 2

perform the multiplication of two matrices

Build Status

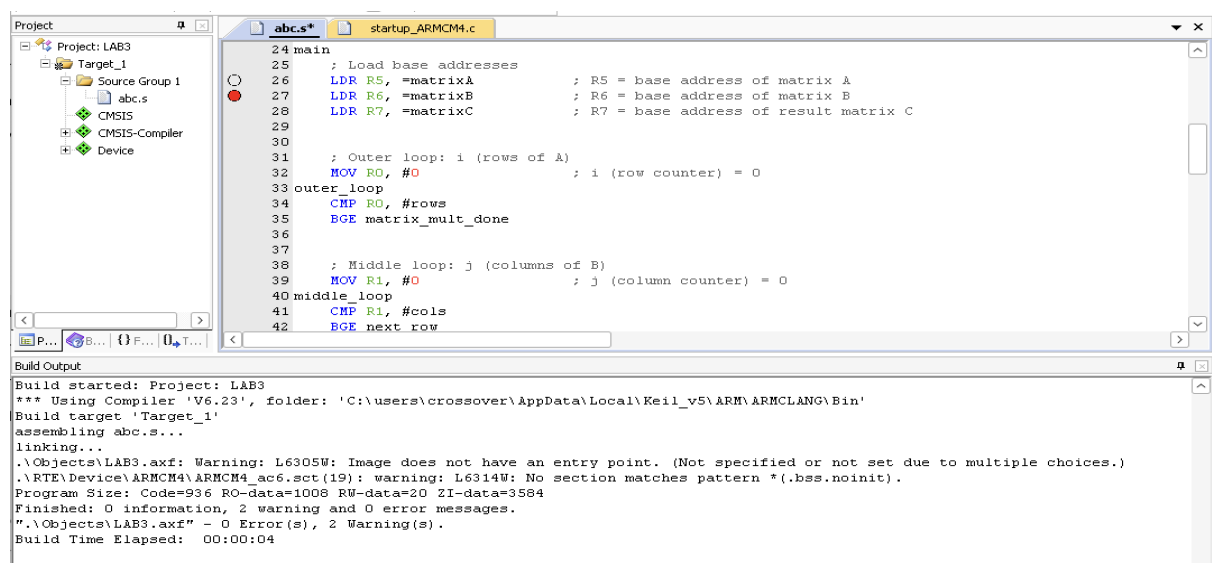


Figure 3: State of the system after the project is built.

Register Status After Execution

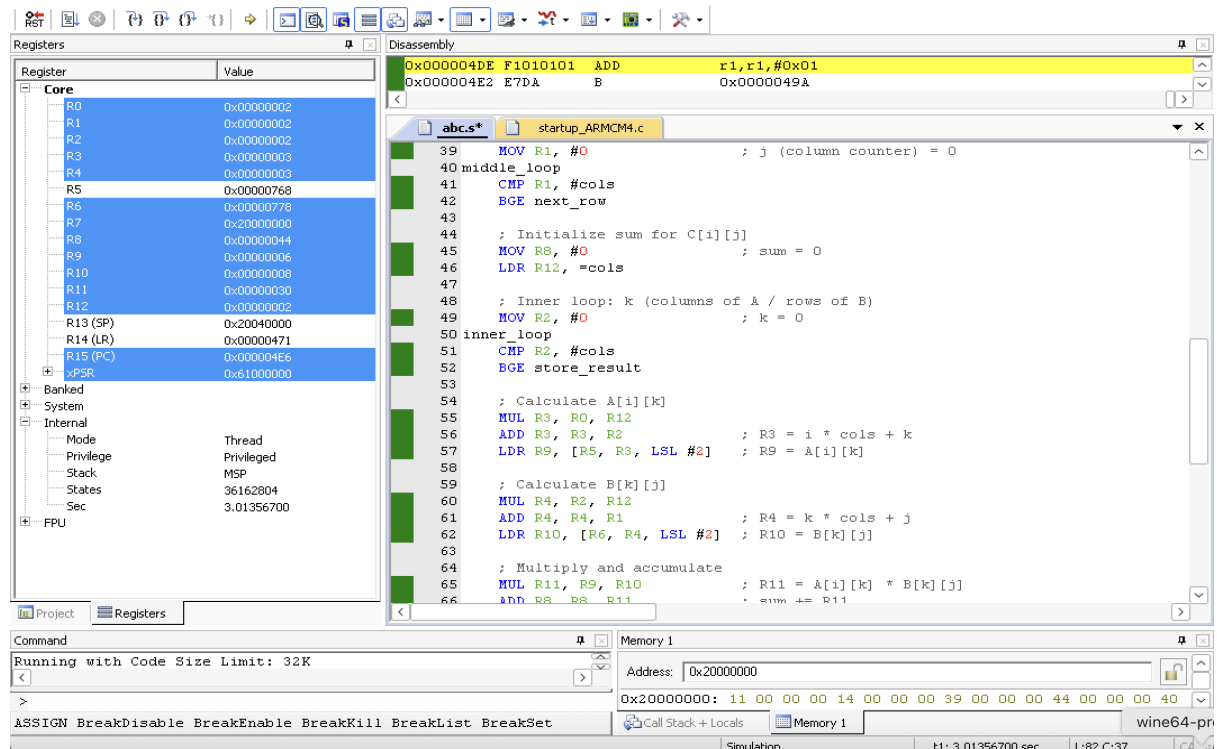


Figure 4: State of the system after the code has been executed.

3 Task 3

Add all data bytes and store any carry generated by calling a function `Addbyte`

Build Status

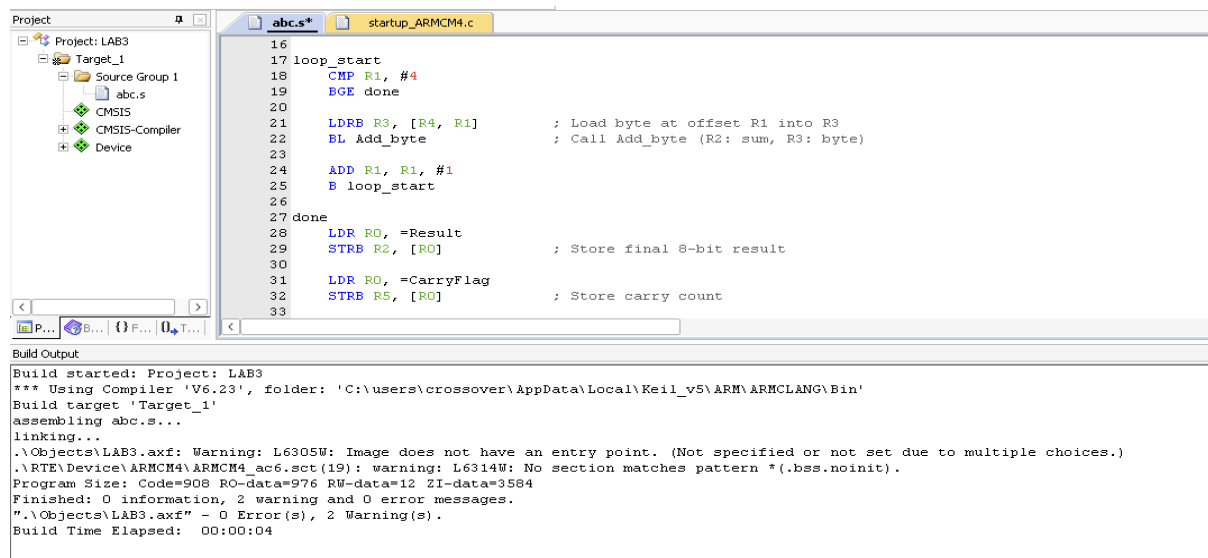


Figure 5: State of the system after the project is built.

Register Status After Execution

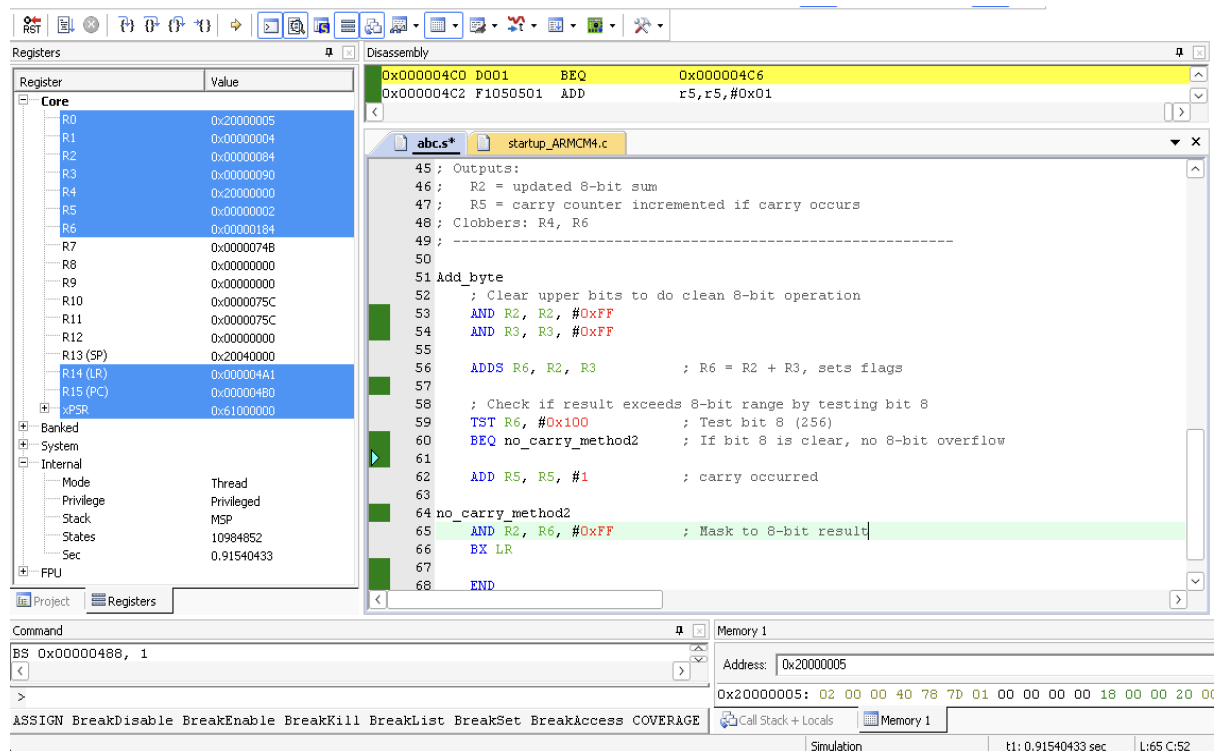


Figure 6: State of the system after the code has been executed.

4 Task 4

Convert BCD data to Binary data by calling a function BCD_{binary}

Build Status

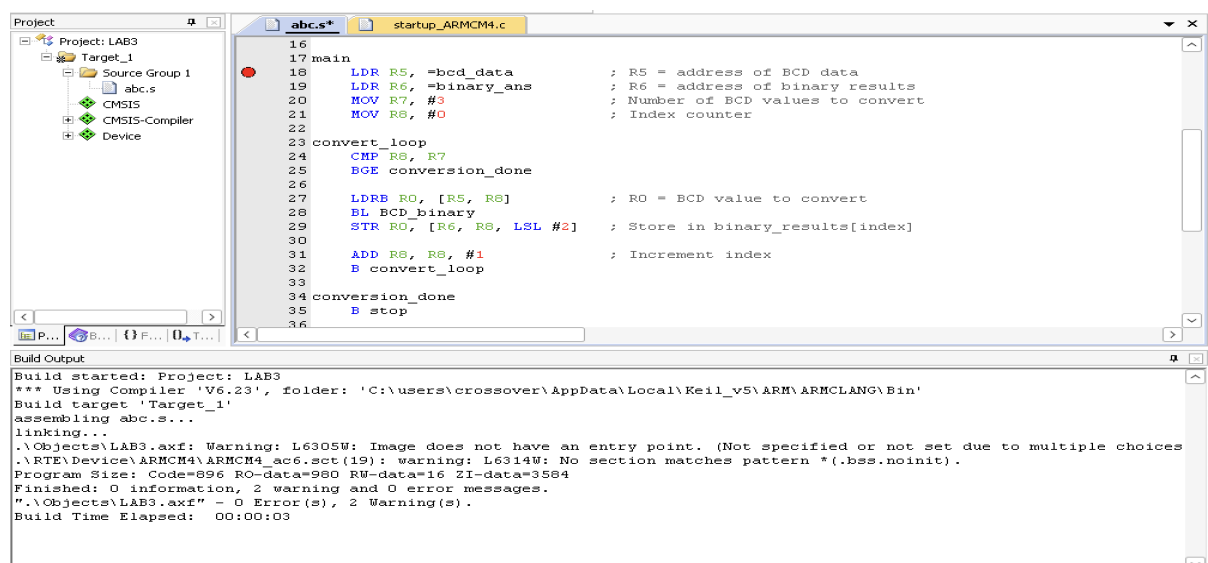


Figure 7: State of the system after the project is built.

Register Status After Execution

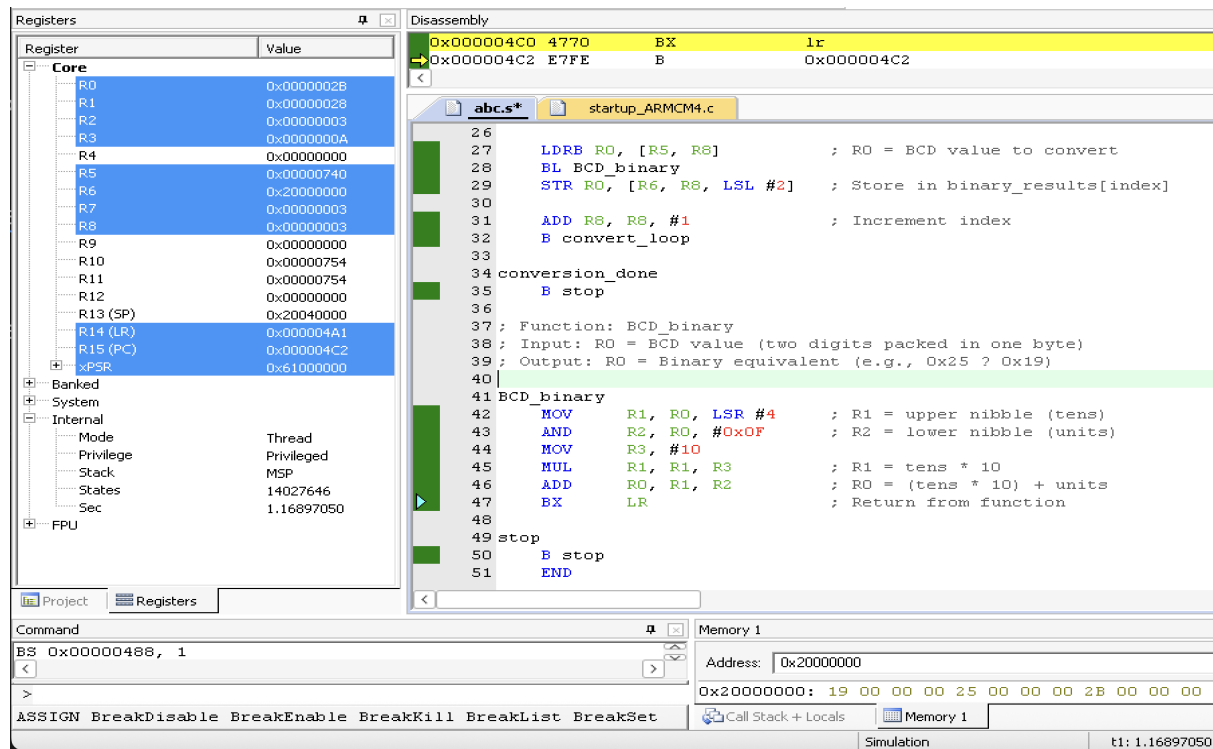


Figure 8: State of the system after the code has been executed.

5 Task 5

Implement a UP- COUNTER to count '00 – 99' in BCD and generate a delay of one second between the counts

Build Status

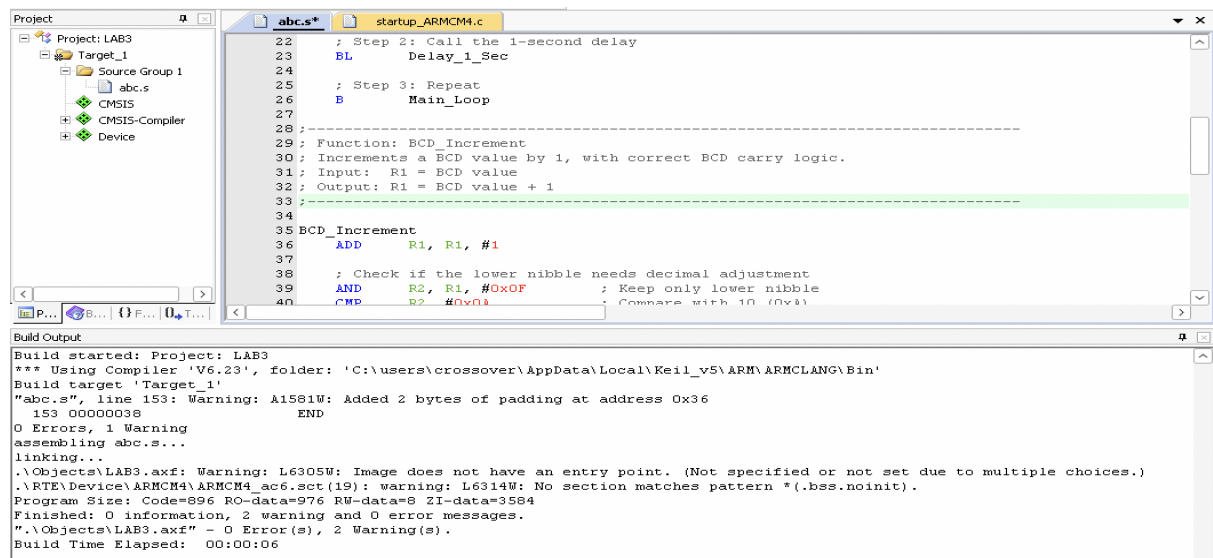


Figure 9: State of the system after the project is built.

Register Status After Execution

The screenshot displays the Keil uVision IDE interface during a simulation. The **Registers** window on the left lists the state of various registers:

Register	Value
R0	0x20000004
R1	0x00000030
R2	0x0000000A
R3	0x00000000
R4	0x017D7840
R5	0x00000000
R6	0x0000041C
R7	0x0000073F
R8	0x00000000
R9	0x00000000
R10	0x00000750
R11	0x00000750
R12	0x00000000
R13 (SP)	0x20040000
R14 (LR)	0x000006F3
R15 (PC)	0x000006E6
xPSR	0x61000000

The **Disassembly** window shows the assembly code being executed, with the following key instructions highlighted:

```

2 CPU_SPEED_MHZ EQU 12 ; Keil Simulator's Default Clock Speed is 12 MHz
3
4 AREA |.data|, DATA, READWRITE
5 BcdCounter DCB 0x00 ; The BCD counter, starts at 0
6
7 AREA |.text|, CODE, READONLY
8 ENTRY
9 EXPORT main
10
11 main
12
13 LDR R0, =BcdCounter ; R0 = Address of the BCD counter
14
15 Main_Loop
16
17 ; Step 1: Increment the BCD counter
18 LDRB R1, [R0] ; Load the current BCD value
19 BL BCD_Increment ; increment R1 in BCD format
20 STRB R1, [R0] ; Store the new BCD value back to memory
21
22 ; Step 2: Call the 1-second delay
23 BL Delay_1_Sec
24
25 ; Step 3: Repeat
26 B Main_Loop
27
28 ;-----
29 ; End of BCD Increment

```

The **Command** window at the bottom shows the simulation status: "Running with Code Size Limit: 32K". The **Memory** window shows the address 0x20000004 with the value 30 00 00 00 10 00 00 20 00 00 00 00 00 00 00 00.

Figure 10: State of the system after the code has been executed.